



Week 6 Lecture 2

▼ Class	BSCCS2003
🕒 Created	@October 11, 2021 7:15 PM
🔗 Materials	
# Module #	33
▼ Type	Lecture
☰ Week #	6

REST

REST

- REpresentational State Transfer

What does that mean?

- State information between client and the server explicitly transferred with every communication

Sequence

- Client accesses Resource identifier from the server
 - Usually URI - superset of URL
 - Typically start from home page of the application
 - No initial state assumed
- Resource Operation specified as part of access
 - If HTTP, then GET,POST, etc.
 - Not fundamentally tied to the protocol
- Server responds with new Resource Identifier
 - New state of system; new links to follow, etc.

State of interaction transferred back and forth

HTTP

- One possible protocol to carry the REST messages

- Use the HTTP verbs to indicate action
- Standardize some types of functionality

-
- **GET:** Retrieve representation of target resource's state
 - **POST:** Enclose data in request: target resource "processes" it
 - **PUT:** Create a target resource with data enclosed
 - **DELETE:** Delete the target resource

Idempotent Operations

- Repeated application of the operation is not a problem
- **Example:** GET is always safe - read-only operation
- **Example:**
 - **PUT:** will always create the same new resource. If already exists, may give error
 - **DELETE:** can delete only once. May error on repeated deletion, but won't change the data
 - **POST:** May NOT be idempotent
 - **Example:** Add comment to a blog - repeat will cause multiple copies

CRUD

- CRUD: Database operations
- Create, Read, Update, Delete
- Typically a common set of operations needed in most web applications
 - Good candidate of REST based functionality

REST $\neq$ CRUD

But, they do work together quite well

Data Encoding

- **Basic HTML:** for simple reasons
- **XML:** Structured data response
- **JSON:** Simpler form of structured data

Data serialization for transferring complex data types over text based format

JSON

- JavaScript Object Notation
- Nested arrays:
 - Serialize complex data structures like dictionaries, arrays, etc.

```
{
  "firstName": "John",
  "lastName": "Smith",
  "age": 27,
  "address": {
    "streetAddress": "21 2nd Street",
    "postalCode": "10021-3100"
  },
  "phoneNumbers": [
    {
      "type": "home",
      "number": "212 555-1234"
    },
    {
      "type": "office",
      "number": "646 555-4567"
    }
  ],
  "children": [],
  "spouse": null
}
```

API data transfer format

- Input to API: text - HTTP
- Output: complex data types - JSON, XML, YAML, etc
 - JSON most commonly used
- Different from internal server representation
- Different from final view presentation

YAML

- **Yet Another Markup Language:** common alternative, especially for documentation and configuration